Cloud Computing e-Portfolio: Reflections on Weeks 1 to 12

Introduction

This e-Portfolio brings together what I learned over the twelve weeks of the Cloud Computing module. The module started with the basic building blocks, the three service models of Infrastructure as a Service, Platform as a Service and Software as a Service, and the four deployment models of public, private, community and hybrid cloud (Mell and Grance, 2011). From there it moved to planning cloud adoption with frameworks such as TOGAF and ROCCA (Anggraini, Binariswanto and Legowo, 2019), and then into practical work on automation, containers, security, disaster recovery and migration. It finished with a cloud strategy for a hypothetical reinsurer and a look at emerging trends such as AI, edge and quantum computing.

To keep my reflections from becoming a list of tasks, I loosely followed the “What? So what? Now what?” model from Rolfe, Freshwater and Jasper (2001). For each topic I asked what I did, what it meant for me and for IT practice, and what I would do differently. The scripts, reports and discussion posts referred to below are all available in my GitHub e-Portfolio https://donaldduodu-essex.github.io/e-Portfolio/ .

Weekly Reflections

1. Service models and choosing a provider (Units 1 and 3)

In Unit 1 I compared AWS and Google Cloud, and in Unit 3 I looked at IaaS, PaaS and SaaS in more depth. Before this, I thought of the service models as different products. Working through them changed that, and I now see them as a decision about how much control an organisation is willing to give up in exchange for convenience. What surprised me was the market data. SaaS still has the largest share, but IaaS is growing fastest, mainly because of the computing demand created by AI workloads (Grand View Research, 2026). For IT practice, this means choosing a provider is less about headline price and more about which ecosystem fits the organisation’s skills and long term direction, which was the conclusion I reached in my Unit 1 post.

2. Automation and Infrastructure as Code (Units 4 and 5)

In Unit 4 I wrote a Bash script that provisioned an Azure virtual machine and configured it automatically, and in Unit 5 I used the Azure CLI to create a storage account and a private blob container. The biggest lesson came from something small. My storage account name kept failing because Azure requires it to be globally unique, lowercase and within a character limit, which I only found out when I ran the script against a live environment. That showed me the difference between code that looks right and code that has actually been tested. Wiggins (2017) argues that environments should be kept as similar as possible by treating setup as code, and Morris (2020) adds that Infrastructure as Code makes changes reviewable and

version controlled. I now see a script as documentation and an audit trail, not just a time saver, and this idea carried directly into my use of Terraform and Ansible in Units 10 and 11.

3. Containers and Kubernetes (Units 3 and 5)

In Unit 3 I chose Kubernetes as my cloud design tool because it is portable across providers, and in Unit 5 I deployed a Twelve Factor app on a local Minikube cluster, using a ConfigMap to keep configuration out of the container image. The peer feedback on this task taught me the most. Stelios pointed out that my YAML indentation had not carried over cleanly, which matters because Kubernetes manifests break if indentation is wrong. Moza asked whether I had tested what happened when a pod was terminated, and I had to admit I had not. I had written about the disposability principle without actually proving it. Hamzaoui et al. (2024) note that containers solve many portability problems but bring their own complexity around orchestration, and I experienced that directly. In future I want to test failure as well as success.

4. Hybrid cloud and security (Unit 7)

Unit 7 covered two linked topics. I compared AWS Outposts with Azure Arc, and I carried out a security audit of a Docker application using Greenbone Community Edition, mapping the findings to ISO/IEC 27001 controls. Most of the issues I found, such as an outdated base image and a container running as root, came from default settings that had never been changed. Reading a classmate’s audit also showed me that a network scanner cannot see inside an image or a Dockerfile, so a clean scan can give a false sense of security. Sharma and Kumar (2024) explain that keeping security governance consistent across mixed environments is one of the main challenges of hybrid cloud. For modern IT practice, I think this supports moving security checks earlier into the deployment pipeline, rather than relying on audits after the event.

5. Resilience, disaster recovery and migration (Units 8 and 9)

In Unit 8 I wrote a disaster recovery plan for an Azure web application using Restic, with a recovery time objective of thirty minutes and a recovery point objective of sixty minutes. My simulated restore took around eight minutes, but forty minutes of data were lost, which was much closer to the limit than I expected. Carson and Rodrick (2024) separate platform availability from data recoverability, and this exercise made that distinction real for me, because the cloud provider being up did not stop my table from being deleted. In Unit 9 I migrated a MySQL database to Azure and found that Flexible Server would not accept my XtraBackup physical backup, so I had to switch to mysqldump with replication. Dunbar (2024) warns about the lock in that comes from relying on a provider’s own tools. Looking back, lock in kept coming up

in almost every unit, and it has become one of the main things I now look for when assessing a cloud design.

Skills Development

My technical skills developed gradually across the module, with each task building on the one before. I started with Bash and the Azure CLI, which gave me the basics of provisioning resources through code. This prepared me for the Northbridge Re report, where I used Terraform to create AWS resources and Ansible to configure servers. Splitting the work between the two tools made sense to me because I had already seen the difference between creating infrastructure and configuring it in the earlier Azure tasks.

With containers, I learned to write a Dockerfile, build and run images, start a Minikube cluster and manage deployments with kubectl. I also used Greenbone to scan containers and Restic to back up and restore a database to a separate region. The migration task gave me practical experience with mysqldump, Azure Blob Storage and MySQL replication, and it taught me how important it is to record the correct binary log position, because a mistake there would leave a silent gap in the data.

The module also developed my teamwork. In the group project on Azure Resource Manager, our team of four agreed to use Microsoft Teams and SharePoint, limited each section to 250 words and split the work by topic, and my section was the real world case study. Coordinating with others made me more careful about writing in a way that could fit alongside other people's sections.

Being honest, I am most comfortable with Bash and the Azure CLI, and less confident with Kubernetes networking and with writing tests for failure scenarios. I also need to get better at saving screenshots and evidence as I go, rather than trying to recreate them afterwards.

Application to Industry: Reinsurance

For the industry section, I focused on reinsurance, which I also used in my Northbridge Re strategy report. Reinsurers have a very specific computing pattern. Catastrophe modelling needs large amounts of computing power for a few weeks around renewals or after a major event, and very little for the rest of the year. This makes the elastic, pay as you go model of the cloud a strong fit, and in my report I proposed moving these workloads to AWS first using Spot Instances, since model runs can be stopped and restarted.

However, I do not think the technology is the deciding factor in this sector. Reinsurers are heavily regulated, and the Prudential Regulation Authority treats

significant cloud use as outsourcing, expecting firms to manage concentration risk and to have a tested exit plan (Prudential Regulation Authority, 2021). The UK has gone further by creating a regime for critical third parties, which recognises that a failure at one large cloud provider could affect many financial firms at once (Bank of England, Prudential Regulation Authority and Financial Conduct Authority, 2024). Carugati (2023) adds that the largest cloud providers are also becoming the main suppliers of generative AI, which could increase this concentration further.

This is where I had to be critical of my own proposal. I argued that Infrastructure as Code helps with exit planning, but my Terraform code was written specifically for AWS resources and would need rewriting for another provider. Only the Ansible playbooks would move easily. The cost case also has limits, because savings from Spot Instances can be offset by data transfer charges and the cost of building new skills. Overall, I believe cloud computing fits reinsurance well, but only if governance, exit planning and data protection are designed in from the beginning rather than added later.

Future Learning Goals

My plan for further development builds on the gaps I identified above and on the emerging technologies covered in weeks 10 to 12.

In the next three months, I plan to take the AWS Certified Cloud Practitioner exam, then start preparing for the HashiCorp Terraform Associate certification. As a practical goal, I want to deploy a small version of the Northbridge Re design in a free tier AWS account, so that my Terraform code is actually tested rather than just written.

Between three and six months, I want to improve my Kubernetes skills by testing failure scenarios, such as deleting pods and checking they are replaced, and by adding a container image scanner such as Trivy to a simple pipeline. This would close the two gaps from Units 5 and 7.

Between six and twelve months, I want to explore the emerging areas from the final units. For edge computing, I plan to read further on how AI is being used to secure fog and edge environments (Alsadie, 2024; Shi et al., 2016), since this links security, which interests me most, with a growing trend. For quantum computing, I will work through the beginner Qiskit tutorials (Javadi-Abhari et al., 2024), treating this as building awareness rather than a specialism, since Alshareef (2023) presents quantum as a longer term development. I also want to understand post quantum cryptography, because NIST has already published its first standards (National Institute of Standards and Technology, 2024) and this will affect any organisation that keeps data for many years.

I will track my progress by adding each completed task and certificate to my e-Portfolio, which will also help me keep better evidence than I did during this module.

Conclusion

Looking back over the twelve weeks, the biggest change in my thinking is that cloud computing is not only a technical topic. Each practical task taught me a tool, but the reflections and peer feedback taught me to question my own work, especially around testing, security defaults and lock in. These are the areas I will carry into my future learning.

References

Alsadie, D. (2024) 'Artificial intelligence techniques for securing fog computing environments: trends, challenges, and future directions', IEEE Access.

Alshareef, H.N. (2023) 'Current development, challenges, and future trends in cloud computing: a survey', International Journal of Advanced Computer Science and Applications, 14(3), pp. 329 to 338.

Anggraini, N., Binariswanto and Legowo, N. (2019) 'Cloud computing adoption strategic planning using ROCCA and TOGAF 9.2: a study in government agency', Procedia Computer Science, 161, pp. 1247 to 1254.

Bank of England, Prudential Regulation Authority and Financial Conduct Authority (2024) PS16/24: Operational resilience: critical third parties to the UK financial sector. London: Bank of England.

Carson, R. and Rodrick, M. (2024) Business continuity and disaster recovery. Birmingham: Packt Publishing.

Carugati, C. (2023) Competitive relationship between cloud computing and generative AI. Brussels: Bruegel.

Dunbar, W. (2024) Vendor locked DevOps strategies. New York: Association for Computing Machinery.

Grand View Research (2026) Cloud computing market size and share report, 2026 to 2033. Available at: https://www.grandviewresearch.com/industry-analysis/cloud-computing-industry (Accessed: 08 October 2026).

Hamzaoui, I. et al. (2024) 'A topical review on container based cloud revolution: multi directional challenges, and future trends', SN Computer Science, 5(1), pp. 1 to 20.

Javadi-Abhari, A. et al. (2024) Quantum computing with Qiskit. arXiv preprint arXiv:2405.08810.

Mell, P. and Grance, T. (2011) The NIST definition of cloud computing. NIST Special Publication 800-145. Gaithersburg, MD: National Institute of Standards and Technology.

Morris, K. (2020) Infrastructure as code: dynamic systems for the cloud age. 2nd edn. Sebastopol, CA: O'Reilly Media.

National Institute of Standards and Technology (2024) FIPS 203: module lattice based key encapsulation mechanism standard. Gaithersburg, MD: NIST.

Prudential Regulation Authority (2021) SS2/21: outsourcing and third party risk management. London: Bank of England.

Rolfe, G., Freshwater, D. and Jasper, M. (2001) Critical reflection for nursing and the helping professions: a user's guide. Basingstoke: Palgrave Macmillan.

Sharma, S.K. and Kumar, P.B.S. (2024) 'Hybrid IT and multi cloud: an emerging trend and improved deployment model', SN Computer Science, 5(1), pp. 1 to 15.

Shi, W., Cao, J., Zhang, Q., Li, Y. and Xu, L. (2016) 'Edge computing: vision and challenges', IEEE Internet of Things Journal, 3(5), pp. 637 to 646.

Wiggins, A. (2017) The twelve factor app. Available at: https://12factor.net (Accessed: 29 September 2026).